■ 2.1.5 Applying Functions to Parts of Expressions

In an expression like `f[{a, b, c}]`, the function `f` is applied to the *whole* of the list `{a, b, c}`. Often you will instead want to apply the function separately to *each* element in a list. You can do this using `Map`.

This applies `f` separately to each element in the list.

```
In[1]:= Map[f, {a, b, c}]
Out[1]= {f[a], f[b], f[c]}
```

This applies `f` to the whole list `{a, b, c}`.

```
In[2]:= f[{a, b, c}]
Out[2]= f[{a, b, c}]
```

There are, however, some mathematical functions in *Mathematica* that automatically apply themselves to each element in a list.

```
In[3]:= Log[{a, b, c}]
Out[3]= {Log[a], Log[b], Log[c]}
```

You can use `Map` to apply a function to parts of any expression, not just lists.

This applies `f` to each element of the sum.

```
In[4]:= Map[f, a + b + c]
Out[4]= f[a] + f[b] + f[c]
```

`Factor` does not work its way inside the g.

```
In[5]:= Factor[ g[-1 + x^2, -1 + x^3] ]
                   2         3
Out[5]= g[-1 + x , -1 + x ]
```

You can use `Map` to apply `Factor` separately to each argument of g.

```
In[6]:= Map[Factor, %]
                                                     2
Out[6]= g[(-1 + x) (1 + x), (-1 + x) (1 + x + x )]
```

`Map[`*f*`,` *expr*`]` applies *f* to the "first level" of parts in *expr*. You can use `MapAll[`*f*`,` *expr*`]` to apply *f* to *all* the parts of *expr*.

This defines a 2×2 matrix `m`.

```
In[7]:= m = {{a, b}, {c, d}}
Out[7]= {{a, b}, {c, d}}
```

`Map` applies `f` to the first level of `m`, in this case the rows of the matrix.

```
In[8]:= Map[f, m]
Out[8]= {f[{a, b}], f[{c, d}]}
```

`MapAll` applies `f` at *all* levels in `m`. If you look carefully at this expression, you will see an `f` wrapped around every part.

```
In[9]:= MapAll[f, m]
Out[9]= f[{f[{f[a], f[b]}], f[{f[c], f[d]}]}]
```

You can use `Map` and `MapAll` to apply either standard built-in *Mathematica* functions, or functions that you have defined yourself.

This defines a function of one argument which takes the first two elements of a list.

```
In[10]:= take2[list_] := Take[list, 2]
```

This applies `take2` to each sublist.

```
In[11]:= Map[take2, {{a, b, c}, {c, a, b}, {c, c, a}}]
Out[11]= {{a, b}, {c, a}, {c, c}}
```

Sometimes you may find yourself defining a new function just in order to be able to use its name in `Map` or `MapAll`. Section 2.1.12 will show how to avoid this by using "pure functions" which do not have explicit names.

`Map[`*f*`,` *expr*`]`	apply *f* to each part of the first level in *expr*
`MapAll[`*f*`,` *expr*`]`	apply *f* to all parts in *expr*

Applying functions to parts of expressions.